Pós Graduação
Data Science e Big Data na Indústria

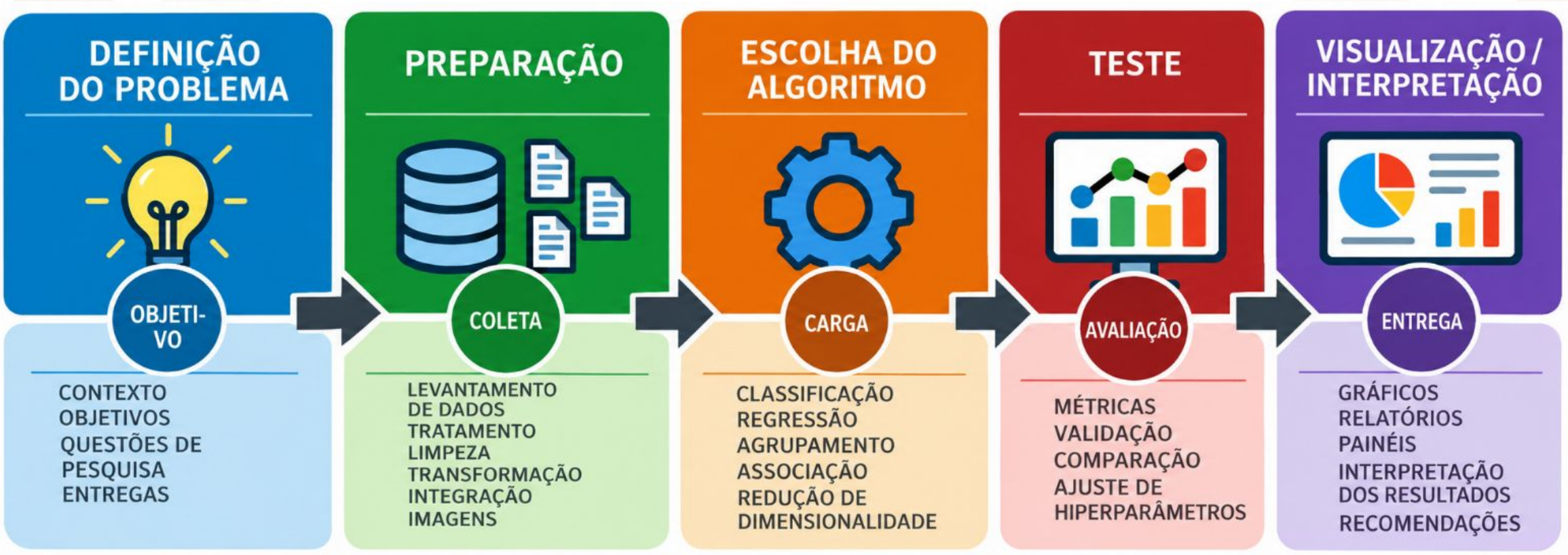
Disciplina: Data Science e Big Data

Aula 03

Prof. Me Daniel Vieira

UNISENAI
Inovação no Ensino Superior para a Indústria

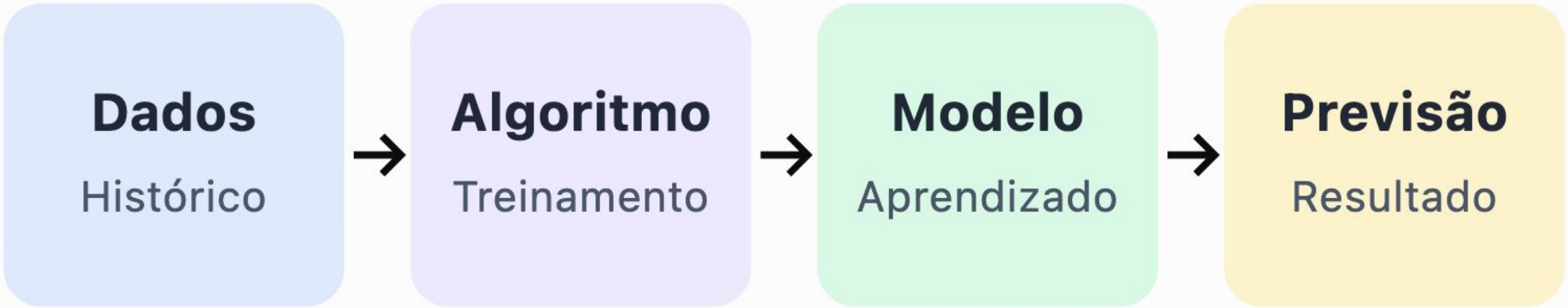
Etapas de um projeto de Data Science



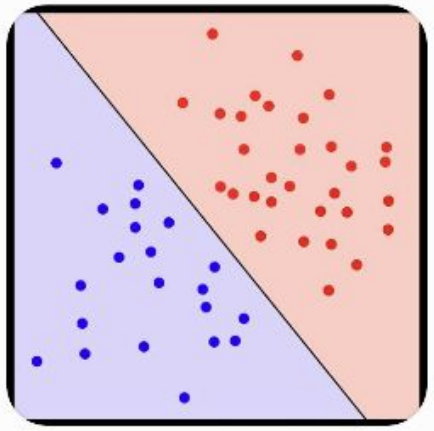
Machine Learning

Machine Learning (Aprendizado de Máquina) é uma área da Inteligência Artificial (IA) que permite aos computadores aprender padrões a partir de dados e utilizar esses padrões para realizar previsões, classificações ou identificar comportamentos, sem que todas as regras sejam programadas explicitamente.

Como funciona o Machine Learning ?



Tipos de Aprendizado de máquina

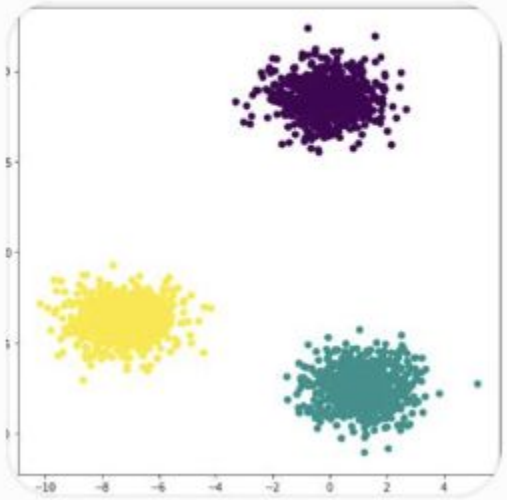


Aprendizado supervisionado

Os dados possuem respostas conhecidas (rótulos). O algoritmo aprende a prever essas respostas.

Exemplo: classificar máquinas como eficientes ou ineficientes.

Tipos de Aprendizado de máquina



Aprendizado não supervisionado

Os dados não possuem respostas previamente definidas.
O algoritmo procura padrões ou agrupamentos.

Exemplo: agrupar máquinas com consumos energéticos semelhantes.

Tipos de Aprendizado de máquina



Aprendizado por reforço

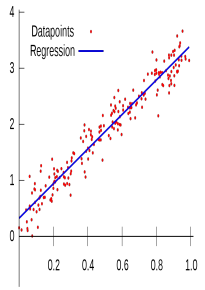
Um agente aprende tomando decisões e recebendo recompensas ou penalidades.

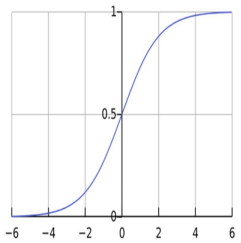
Exemplo: um robô aprender a percorrer um ambiente evitando obstáculos.

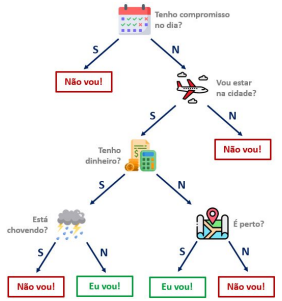
Principais algoritmos de aprendizado de máquina

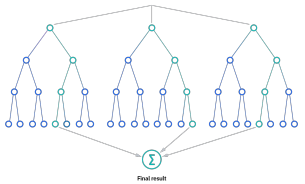
Algoritmos Machine Learning

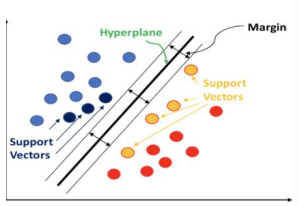
- Regressão linear
- Regressão logística
- Árvore de decisão
- Random Forest
- Support Vector Machine
- K-Means
- Redes neurais artificiais

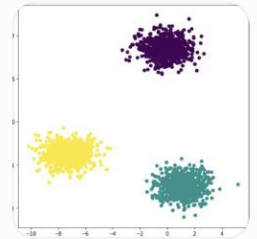


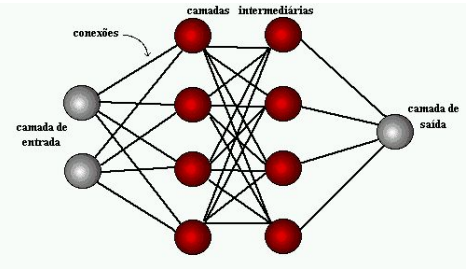












Aplicações

- Prever consumo de energia
- Classificar operação eficiente ou ineficiente
- Identificar máquinas com risco de falha
- Classificar falhas utilizando diversos atributos
- Manutenção preditiva
- Agrupar máquinas por comportamento
- Identificar automaticamente por visão computacional peças com rachaduras ou deformações.

UNISENAI Inovação no Ensino Superior para a Indústria

Imagine uma indústria que deseja identificar se uma máquina está operando de maneira eficiente.

	producao	energia	eficiente
0	1000	200	1
1	500	250	0
2	1200	240	1
3	400	300	0
4	1100	220	1

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

```
#1. Importação das bibliotecas
# Biblioteca para manipulação de dados
import pandas as pd
# Biblioteca para criação de gráficos
import matplotlib.pyplot as plt

# Algoritmo de Árvore de Decisão e sua visualização
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Algoritmo Random Forest
from sklearn.ensemble import RandomForestClassifier

# Função para dividir os dados em treinamento e teste
from sklearn.model_selection import train_test_split

# Funções para avaliar os modelos de classificação
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

```
#2. Criação do conjunto de dados
# Criando um conjunto de dados de 20 máquinas industriais
dados = {
    # Quantidade de peças produzidas
    "producao": [
        1000, 500, 1200, 400, 1100,
        300, 950, 450, 1050, 550,
        1300, 600, 1150, 350, 900,
        650, 1250, 480, 1080, 520],
    # Consumo de energia em kWh
    "energia": [
        200, 250, 240, 300, 220,
        280, 210, 270, 230, 310,
        260, 290, 235, 260, 195,
        320, 250, 295, 225, 275
    ],
    # Classificação conhecida de cada máquina
    # 1 = Eficiente
    # 0 = Ineficiente
    "eficiente": [
        1, 0, 1, 0, 1,
        0, 1, 0, 1, 0,
        1, 0, 1, 0, 1,
        0, 1, 0, 1, 0
    ]
}
# Transforma o dicionário em um DataFrame
df = pd.DataFrame(dados)
print(df)#Exibe os dados
```

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

```python
#3. Separação das variáveis X e y
# X representa as características utilizadas pelo modelo
# São chamadas de variáveis independentes ou preditoras
X = df[["producao", "energia"]]

# y representa a classificação que queremos prever
# É chamada de variável-alvo
y = df["eficiente"]
```

```python
#4. Divisão em treinamento e teste
# Divide o conjunto de dados em treinamento e teste
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X,                # Variáveis de entrada
    y,                # Classificações conhecidas
    test_size=0.20,   # Reserva 20% dos dados para teste
    random_state=42,  # Mantém o mesmo sorteio em cada execução
    stratify=y        # Preserva a proporção das classes
)

# Exibe a quantidade de registros de cada conjunto
print("Treinamento:", len(X_treino))
print("Teste:", len(X_teste))
```

```
Treinamento: 16
Teste: 4
```

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

Nesse exemplo:

- `X` representa as características utilizadas pelo algoritmo.
- `y` representa a classificação que desejamos prever.
- `fit()` realiza o treinamento.
- `predict()` realiza uma previsão utilizando o modelo treinado.

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

```python
#5. Treinamento da Árvore de Decisão
# Cria o algoritmo de classificação
modelo_arvore = DecisionTreeClassifier(
    random_state=42
)

# Treina a árvore utilizando somente os dados de treinamento
modelo_arvore.fit(X_treino, y_treino)

# Utiliza a árvore treinada para classificar os dados de teste
previsao_arvore = modelo_arvore.predict(X_teste)

# Exibe as classificações reais e as previstas
print("Valores reais:", y_teste.values)
print("Previsões:", previsao_arvore)
```

```
Valores reais: [0 0 1 1]
Previsões: [0 0 1 1]
```

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

```
#6. Visualização da Árvore de Decisão
# Define o tamanho da figura
plt.figure(figsize=(12, 6))

# Desenha a árvore treinada
plot_tree(
    modelo_arvore,

    # Nomes das variáveis de entrada
    feature_names=["producao", "energia"],

    # Nomes das classes de saída (0 e 1)
    class_names=["Ineficiente", "Eficiente"],

    # Preenche os nós com cores de acordo com as classes
    filled=True,

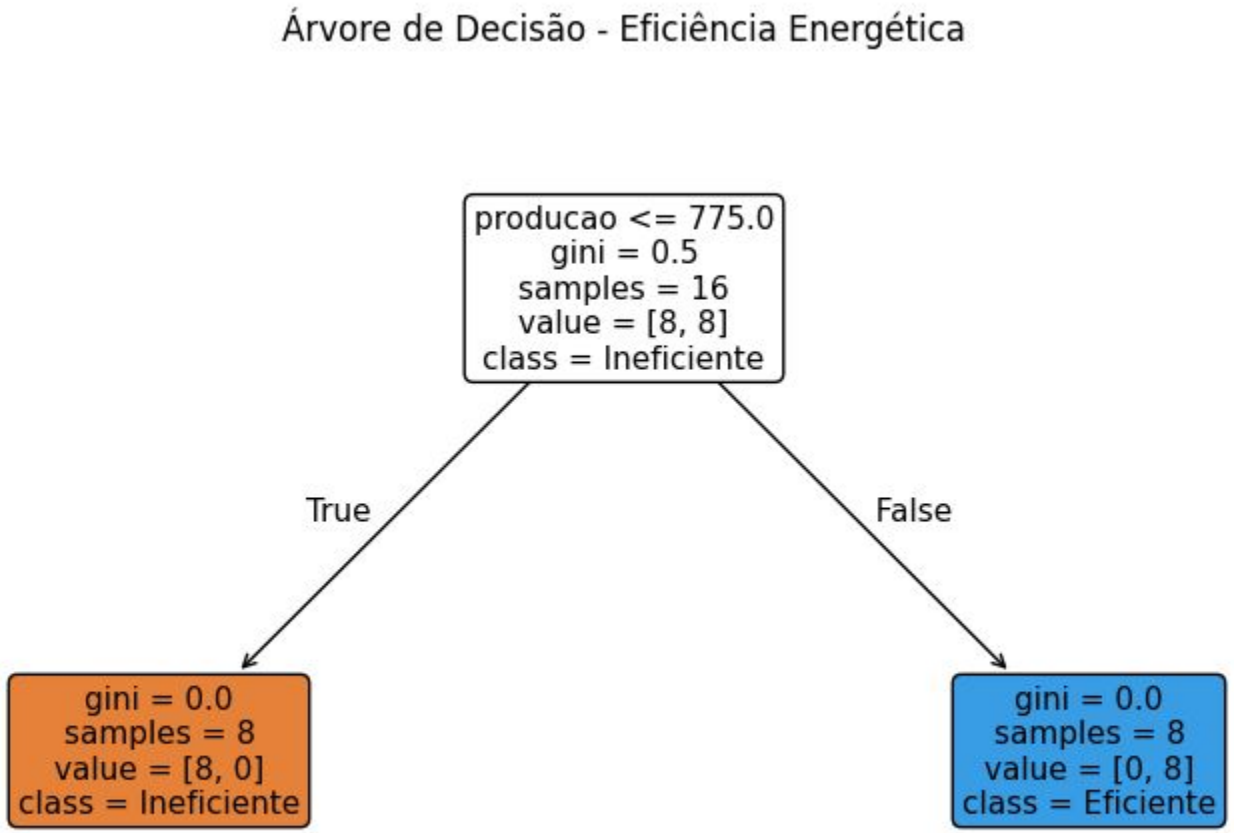
    # Deixa as caixas com bordas arredondadas
    rounded=True,

    # Define o tamanho das letras
    fontsize=11
)

# Adiciona um título
plt.title("Árvore de Decisão - Eficiência Energética")

# Exibe o gráfico
plt.show()
```

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão



Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

A Árvore de Decisão foi treinada com 16 máquinas industriais, utilizando duas variáveis de entrada: produção e consumo de energia. Seu objetivo é classificar cada máquina como eficiente (1) ou ineficiente (0).

1. Nó raiz – Primeira decisão

A primeira condição definida pelo algoritmo foi:

producao <= 775.0

Isso significa que o algoritmo utiliza a quantidade de peças produzidas para separar as máquinas em dois grupos:

- Verdadeiro (True): produção menor ou igual a 775 peças → Ineficiente (0).
- Falso (False): produção maior que 775 peças → Eficiente (1).

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

2. Significado dos valores da árvore

Informação	Interpretação
producao <= 775	Condição utilizada para dividir os dados
gini = 0.5	Nó com máxima impureza para duas classes equilibradas
samples = 16	Quantidade de máquinas utilizadas no treinamento
value = [8, 8]	8 máquinas ineficientes e 8 eficientes
class = Ineficiente	Classe exibida no nó raiz devido ao empate

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

3. Interpretação dos nós finais

Nó esquerdo — Ineficiente

Produção ≤ 775

8 máquinas ineficientes

`gini = 0.0`

Todas as máquinas desse nó pertencem à classe 0.

Nó direito — Eficiente

Produção > 775

8 máquinas eficientes

`gini = 0.0`

Todas as máquinas desse nó pertencem à classe 1.

O índice Gini igual a zero indica que os nós finais são puros, pois contêm máquinas de uma única classe.

4. Exemplo de classificação

Considere uma máquina com:

- Produção: 900 peças.
- Consumo energético: 210 kWh.

Como a produção é superior a 775 peças, a árvore segue pelo caminho da direita e classifica a máquina como **Eficiente (1)**.

Observe que o consumo de energia não foi utilizado nas decisões dessa árvore, pois a produção foi suficiente para separar todas as classes dos dados de treinamento.

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão

Interpretação da Árvore de Decisão

A Árvore de Decisão classificou as máquinas industriais utilizando a variável **produção** como critério de separação.

- **Produção ≤ 775 peças:** máquina classificada como ineficiente (0).
- **Produção > 775 peças:** máquina classificada como eficiente (1).
- **Gini = 0,5:** indica classes equilibradas no nó inicial.
- **Gini = 0,0:** indica separação completa das classes nos nós finais.
- **Samples:** quantidade de registros presentes em cada nó.
- **Value:** distribuição das máquinas entre as classes.

Resultado: o algoritmo identificou uma regra capaz de separar corretamente as 16 máquinas utilizadas no treinamento.

Exemplo 1 de Machine Learning com o algoritmo de Árvore de decisão

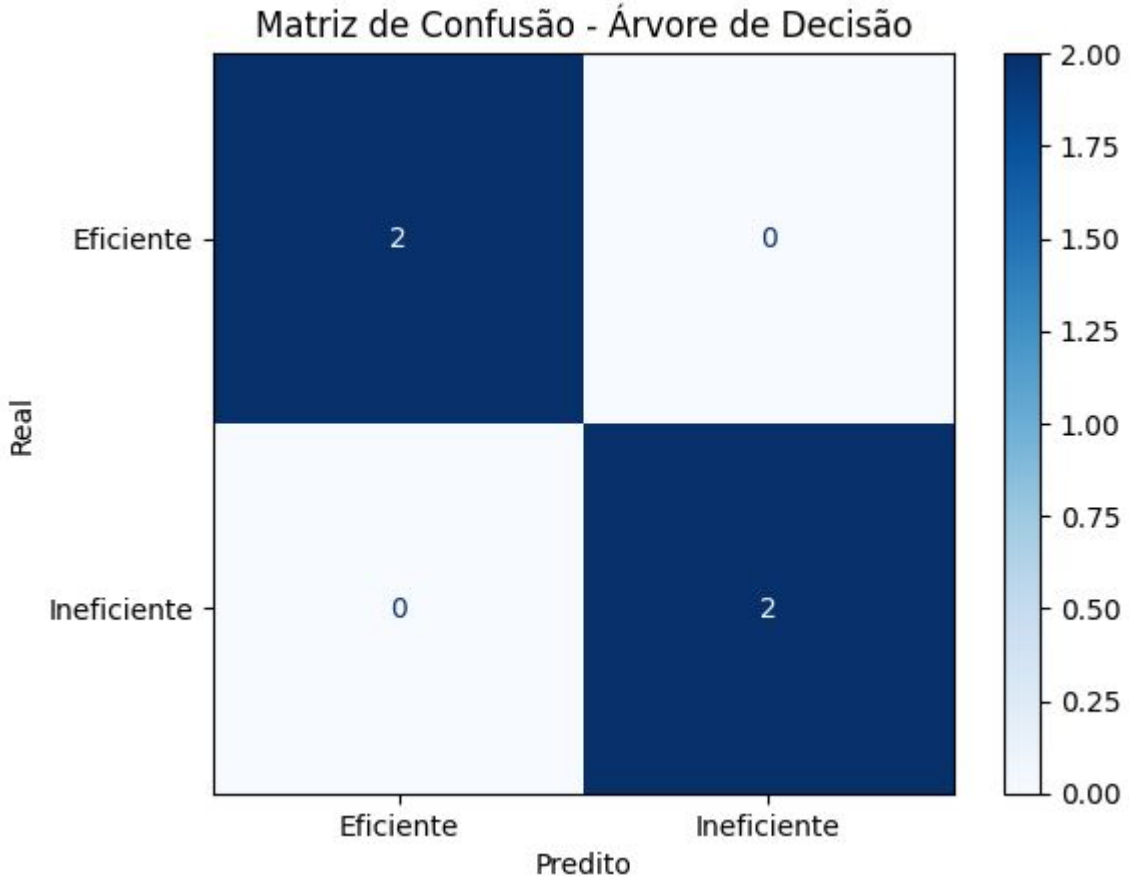
```python
#8. Matriz de confusão
# Calcula a matriz de confusão da Árvore de Decisão
matriz_arvore = confusion_matrix(
    y_teste,           # Classificações reais
    previsao_arvore,   # Classificações previstas
    labels=[0, 1]      # Ordem das classes
)

# Prepara a visualização da matriz
disp = ConfusionMatrixDisplay(
    confusion_matrix=matriz_arvore,
    display_labels=["Eficiente", "Ineficiente"]
)

# Desenha a matriz utilizando a escala de cores azul
disp.plot(cmap="Blues", values_format="d")

# Adiciona o título do gráfico
plt.title("Matriz de Confusão - Árvore de Decisão")
plt.xlabel('Predito')
plt.ylabel('Real')

# Exibe o resultado
plt.show()
```

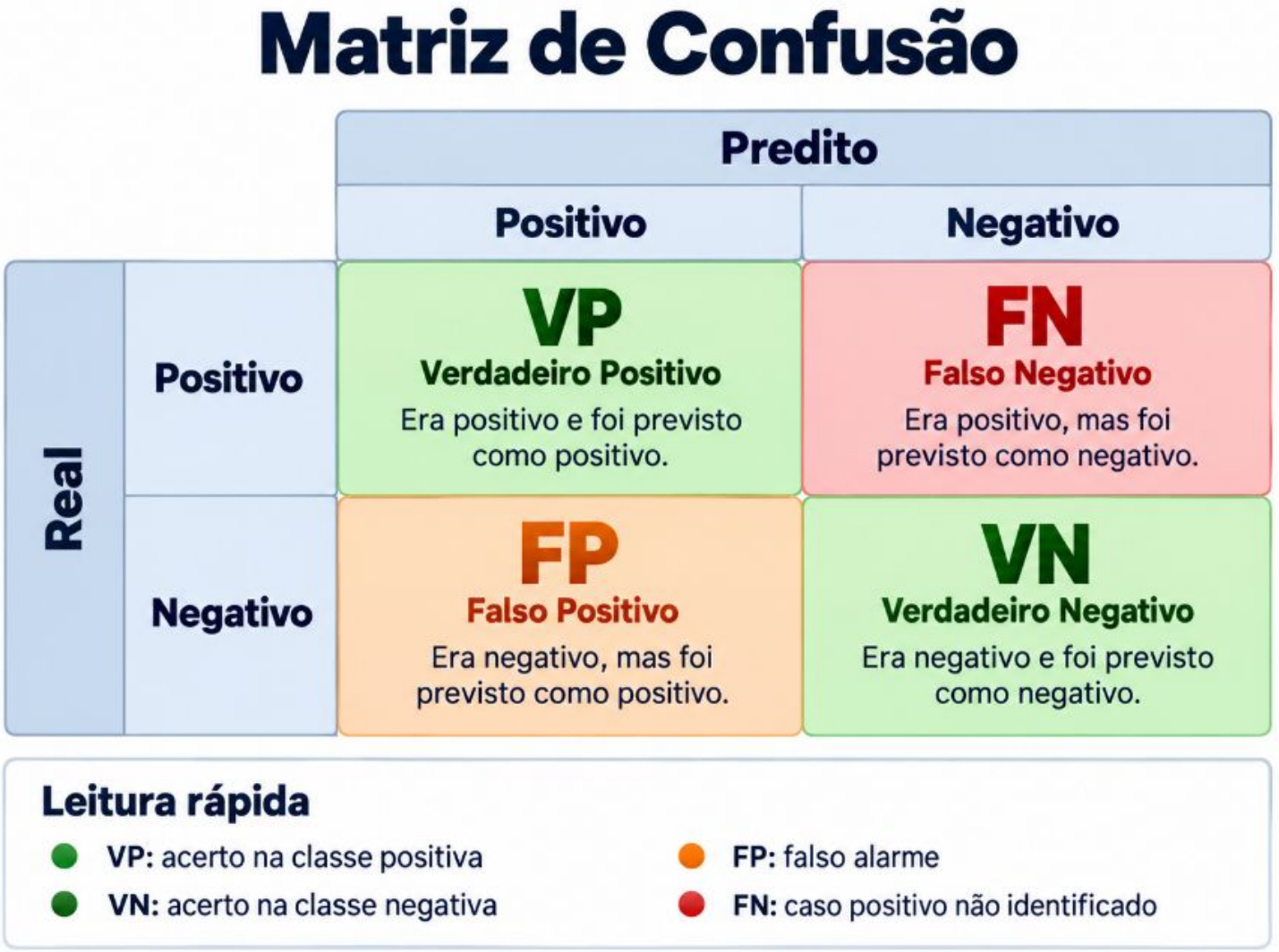


Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão - Métricas do modelo

Os valores são:

- VP (Verdadeiro Positivo) = 2 -Eficientes classificados como eficientes.
- VN (Verdadeiro Negativo) = 2 - Ineficientes classificados como ineficientes.
- FP (Falso Positivo) = 0 - Ineficientes classificados como eficientes.
- FN (Falso Negativo) = 0 - Eficientes classificados como ineficientes.

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão - Métricas do modelo



Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão - Métricas do modelo

1. Acurácia

Mede a proporção de todas as classificações realizadas corretamente.

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}$$

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão - Métricas do modelo

2. Precisão (Precision)

Mede quantas máquinas classificadas como eficientes realmente são eficientes.

$$\text{Precisão} = \frac{VP}{VP + FP}$$

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão - Métricas do modelo

3. Recall (Sensibilidade)

Mede quantas máquinas realmente eficientes foram identificadas corretamente.

$$\text{Recall} = \frac{VP}{VP + FN}$$

Exemplo 1 de Machine Learning com o algoritmo de árvore de decisão - Métricas do modelo

4. F1-Score

Representa a média harmônica entre precisão e recall.

$$F1 = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$$

Exemplo 1 de Machine Learning com o algoritmo de Árvore de decisão

```python
# Calcula as métricas da Árvore de Decisão

# Acurácia: proporção total de classificações corretas
acuracia = accuracy_score(y_teste, previsao_arvore)

# Precisão: entre as máquinas previstas como eficientes,
# quantas realmente eram eficientes?
precisao = precision_score(
    y_teste, previsao_arvore, zero_division=0
)

# Recall: entre todas as máquinas realmente eficientes,
# quantas foram identificadas corretamente?
recall = recall_score(
    y_teste, previsao_arvore, zero_division=0
)

# F1-score: média harmônica entre precisão e recall
f1 = f1_score(
    y_teste, previsao_arvore, zero_division=0
)

# Exibe os resultados das métricas
print(f"Acurácia: {acuracia:.2f}")
print(f"Precisão: {precisao:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-score: {f1:.2f}")
```

Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```
#7. Treinamento do Random Forest
# Cria um modelo Random Forest com 100 árvores de decisão
modelo_rf = RandomForestClassifier(
    n_estimators=100,  # Número de árvores da floresta
    random_state=42   # Garante a reprodução dos resultados
)

# Treina o Random Forest com os dados de treinamento
modelo_rf.fit(X_treino, y_treino)

# Classifica as máquinas do conjunto de teste
previsao_rf = modelo_rf.predict(X_teste)

# Exibe os resultados previstos
print("Previsões Random Forest:", previsao_rf)
```

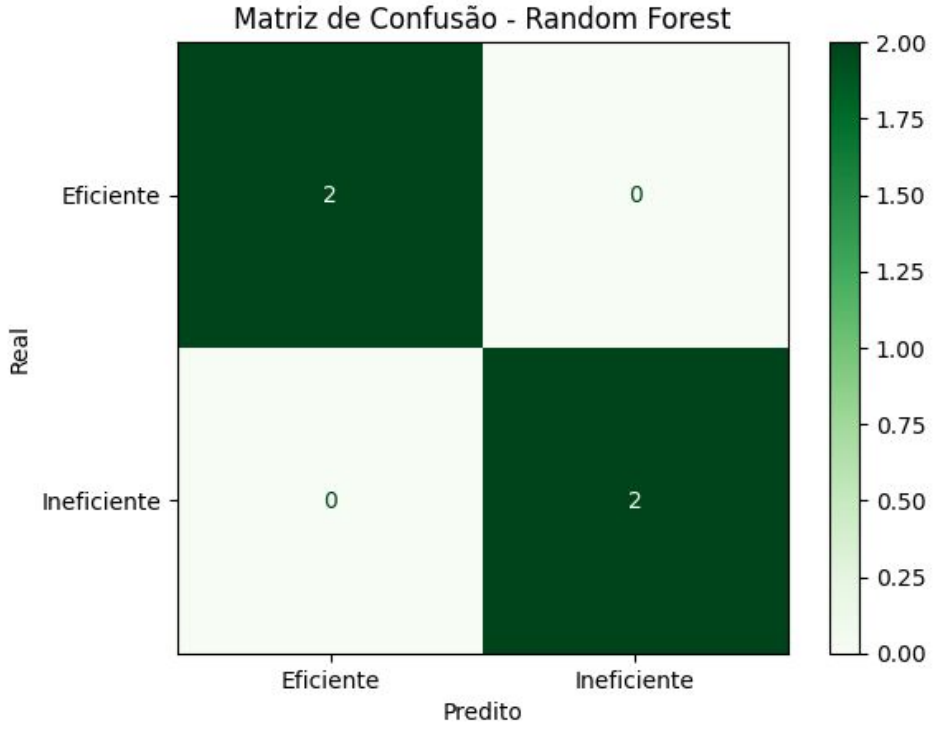
Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```python
#9. Métricas de avaliação
# Calcula a matriz de confusão do Random Forest
matriz_rf = confusion_matrix(
    y_teste,     # Classificações reais
    previsao_rf, # Classificações previstas pelo Random Forest
    labels=[0, 1]
)

# Cria a visualização da matriz
disp = ConfusionMatrixDisplay(
    confusion_matrix=matriz_rf,
    display_labels=["Eficiente", "Ineficiente"]
)

# Exibe a matriz em verde
disp.plot(cmap="Greens", values_format="d")

plt.title("Matriz de Confusão - Random Forest")
plt.xlabel('Predito')
plt.ylabel('Real')
plt.show()
```



Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```python
#10. Comparação dos dois modelos
# Organiza os modelos treinados em um dicionário
modelos = {
    "Árvore de Decisão": modelo_arvore,
    "Random Forest": modelo_rf
}

# Lista para armazenar os resultados das avaliações
resultados = []

# Percorre os dois modelos
for nome, modelo in modelos.items():

    # Realiza previsões com os dados de teste
    previsao = modelo.predict(X_teste)

    # Calcula as métricas e adiciona à lista
    resultados.append({
        "Modelo": nome,
        "Acurácia": accuracy_score(y_teste, previsao),
        "Precisão": precision_score(
            y_teste, previsao, zero_division=0
        ),
        "Recall": recall_score(
            y_teste, previsao, zero_division=0
        ),
        "F1-score": f1_score(
            y_teste, previsao, zero_division=0
        )
```

Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```
32 # Transforma os resultados em um DataFrame
33 comparacao = pd.DataFrame(resultados)
34
35 # Exibe as métricas com duas casas decimais
36 print(comparacao.round(2))
```

	Modelo	Acurácia	Precisão	Recall	F1-score
0	Árvore de Decisão	1.0	1.0	1.0	1.0
1	Random Forest	1.0	1.0	1.0	1.0

Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```python
#11. Previsão de uma nova máquina
# Define as características de uma nova máquina
nova_maquina = pd.DataFrame({
    "producao": [900], # Produção de 900 peças
    "energia": [210]   # Consumo de 210 kWh
})

# Realiza a classificação com os dois algoritmos
for nome, modelo in modelos.items():

    # Obtém a classe prevista pelo modelo
    classe = modelo.predict(nova_maquina)[0]

    # Converte a classificação numérica em texto
    resultado = "Eficiente" if classe == 1 else "Ineficiente"

    # Exibe o resultado
    print(nome, ":", resultado)
```

```
Árvore de Decisão : Eficiente
Random Forest : Eficiente
```

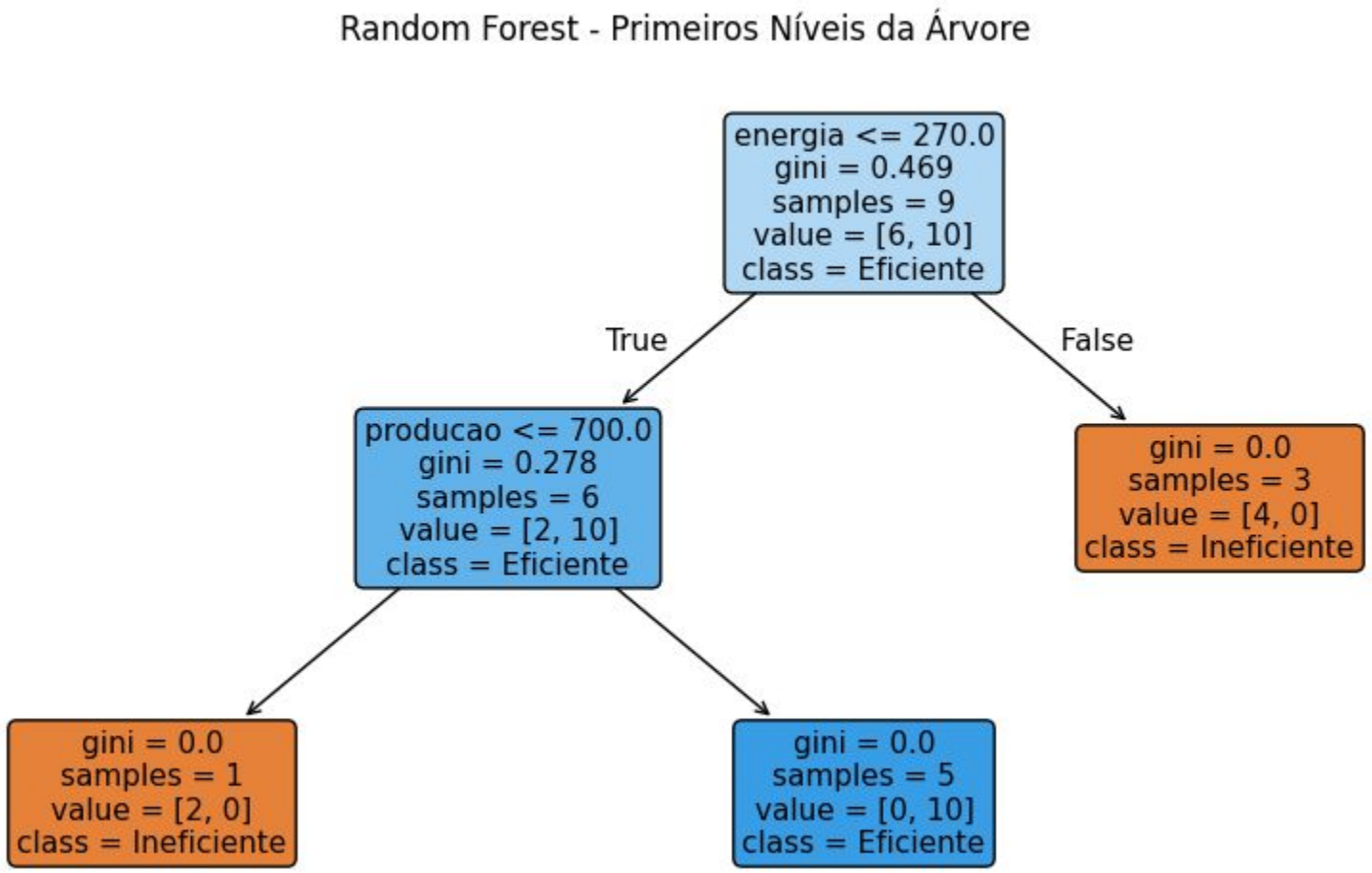
Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```
#13 Visualizar uma árvore do Random Forest
plt.figure(figsize=(12, 6))

plot_tree(
    modelo_rf.estimators_[0],
    feature_names=["producao", "energia"],
    class_names=["Ineficiente", "Eficiente"],
    filled=True,
    rounded=True,
    max_depth=2,  # Mostra somente os primeiros níveis
    fontsize=11
)

plt.title("Random Forest - Primeiros Níveis da Árvore")
plt.show()
```

Exemplo 1 de Machine Learning com o algoritmo de Random Forest



Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
# Cria uma figura com três gráficos
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

# Percorre as três primeiras árvores
for i in range(3):

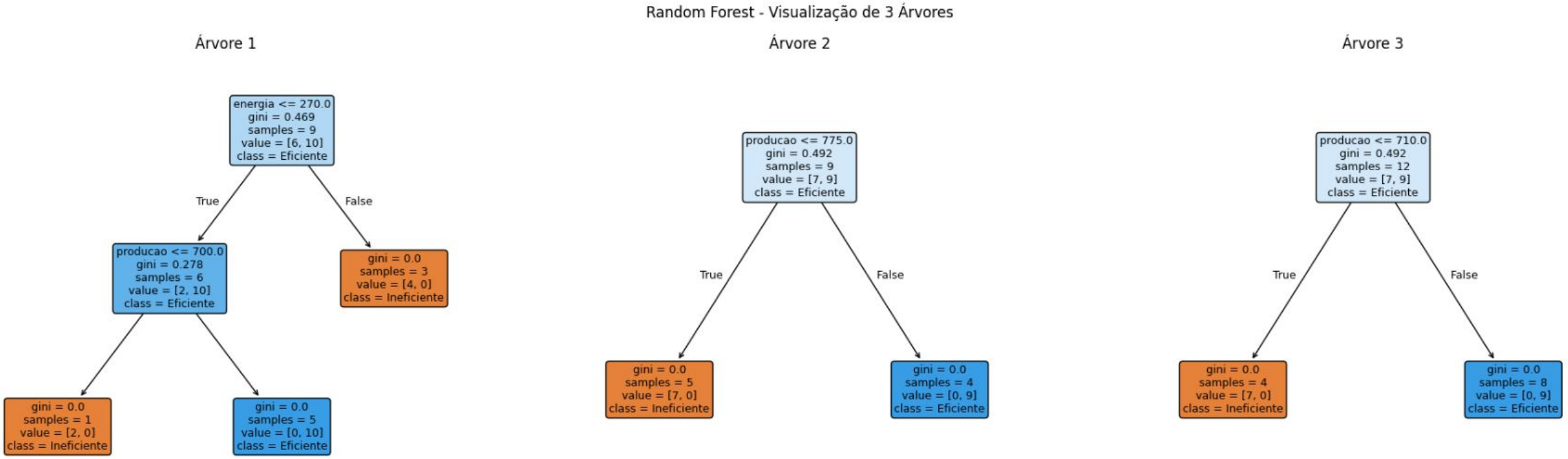
    # Seleciona a árvore correspondente
    arvore = modelo_rf.estimators_[i]

    # Plota a árvore
    plot_tree(
        arvore,
        feature_names=["producao", "energia"],
        class_names=["Ineficiente", "Eficiente"],
        filled=True,
        rounded=True,
        fontsize=9,
        ax=axes[i]
    )

    # Define o título de cada árvore
    axes[i].set_title(f"Árvore {i + 1}")

plt.suptitle("Random Forest - Visualização de 3 Árvores")
plt.tight_layout()
plt.show()
```

Exemplo 1 de Machine Learning com o algoritmo de Random Forest



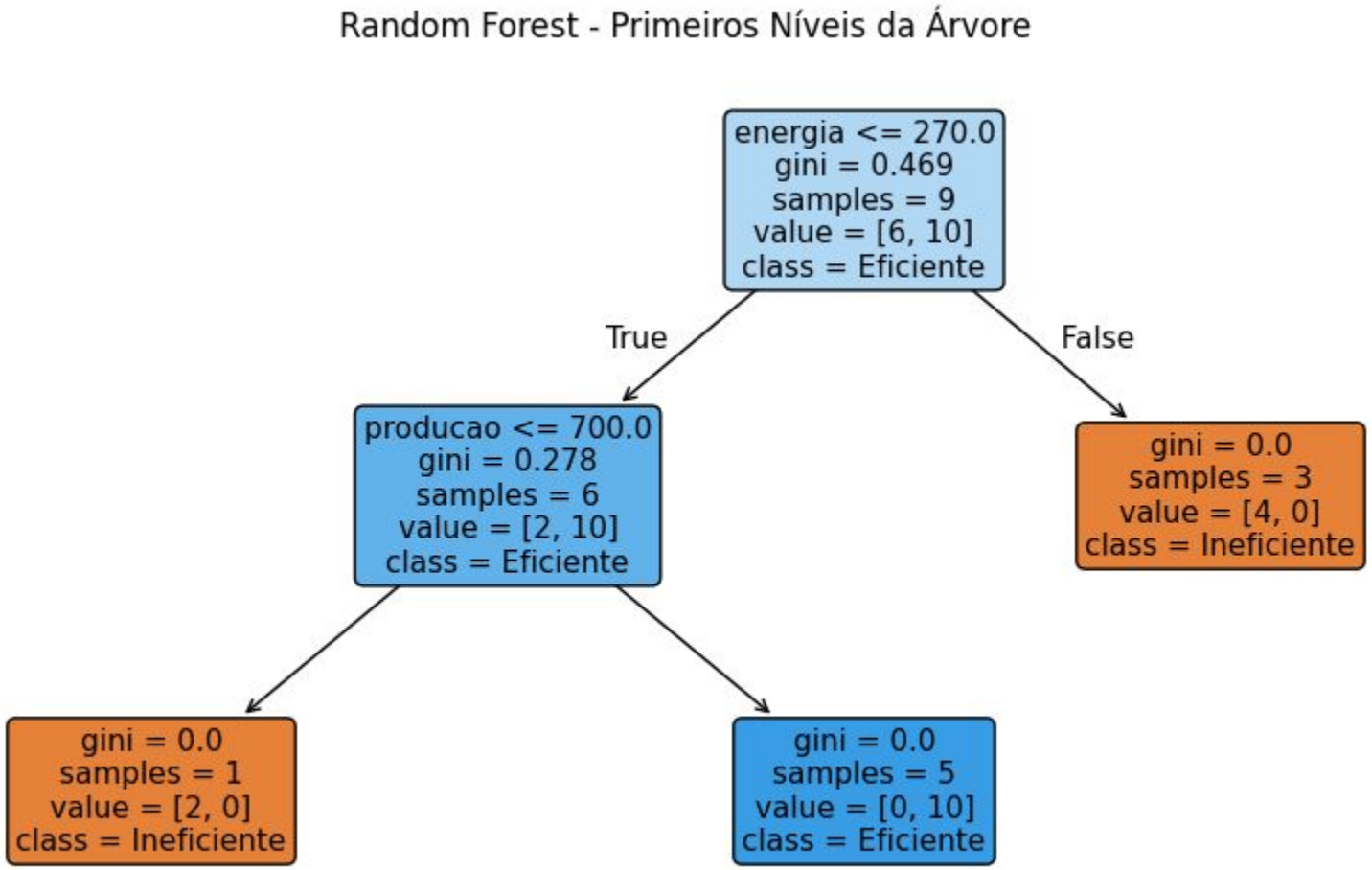
Exemplo 1 de Machine Learning com o algoritmo de Random Forest

```
#15
plt.figure(figsize=(12, 6))

plot_tree(
    modelo_rf.estimators_[0],
    feature_names=["producao", "energia"],
    class_names=["Ineficiente", "Eficiente"],
    filled=True,
    rounded=True,
    max_depth=2,  # Mostra somente os primeiros níveis
    fontsize=11
)

plt.title("Random Forest - Primeiros Níveis da Árvore")
plt.show()
```

Exemplo 1 de Machine Learning com o algoritmo de Random Forest



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Base de dados utilizada

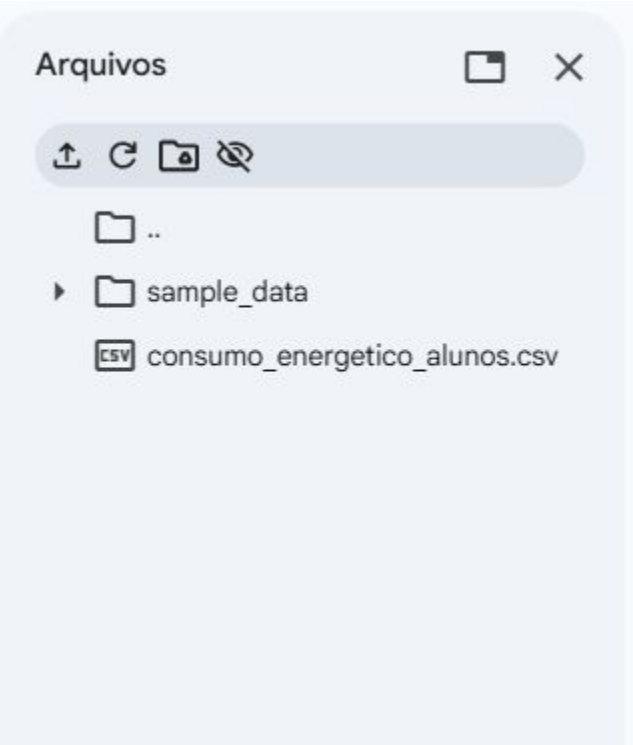
Aula 03 - Pós Data Science e Big Data - Exploração dos dados Objetivos: Detectar problemas de qualidade dos dados, preparar os dados para modelos de Machine Learning, treinar uma Árvore de Decisão, intepretar métricas e aplicar um segundo classificador ao conceito de eficiência energética

```
# Importando bibliotecas
import numpy as np # permite trabalhar com array
import pandas as pd # permite importar a base de dados
import matplotlib.pyplot as plt # Permite plotar gráficos
from sklearn.model_selection import train_test_split # Permite dividir a base de dados em treino e teste
from sklearn.impute import SimpleImputer # Permite tratar dados ausentes na base de dados
from sklearn.pipeline import Pipeline # Permite criar um fluxo nas etapa de analise de dados
from sklearn.compose import ColumnTransformer # Permite aplicar transformações de dados diferentes para colunas diferentes
from sklearn.preprocessing import OneHotEncoder # Permite converter variáveis categóricas em um forma numérico para que os modelos de Machine Learning entendam
from sklearn.tree import DecisionTreeClassifier, plot_tree #Importa o modelo de árvore de decisao
from sklearn.metrics import(accuracy_score, precision_score, recall_score, f1_score,classification_report, ConfusionMatrixDisplay)
import seaborn as sns
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Carregar a base de dados no Google Colab




```
1 # Importando a base de dados
2 df = pd.read_csv('consumo_energetico_alunos.csv')
```

```
1 df.head()
```

	timestamp	turno	producao_unidades	temperatura_ambiente_c	maquinas_ativas	horas_operacao_turno	consumo_kwh
0	2026-10-12 22:00:00	noite	1102.0	21.80	7.0	4.86	283.36
1	2027-11-15 14:00:00	tarde	1356.0	21.31	6.0	6.90	335.73
2	2028-12-27 06:00:00	manha	796.0	28.89	3.0	6.74	197.15
3	2027-04-25 14:00:00	tarde	1406.0	22.52	7.0	6.05	291.80
4	2028-03-27 06:00:00	manha	1431.0	23.98	7.0	5.44	269.41

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
1 print(f"Quantidade de linhas {len(df)} | Quantidade de colunas {len(df.columns)}")
2
```

```
Quantidade de linhas 5050 | Quantidade de colunas 8
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```python
# Exploração dos dados
'''
Quantas linhas e colunas ? Quais variáveis são numéricas ? Quais são as classes de turno ? Há dados ausentes e repetidos ?
'''
print("Tipos de dados")
print(df.dtypes) # Exibe os tipos de dados das colunas
print('\n Ausentes')
print(df.isna().sum()) # Verificando dados ausentes
print('\n Duplicados: ',df.duplicated().sum()) # Verificando dados duplicados
print("\n Distribuição de turnos")
print(df['turno'].value_counts())
display(df.select_dtypes(include="number").describe().round(2))
```

```
Tipos de dados
timestamp                 object
turno                     object
producao_unidades        float64
temperatura_ambiente_c   float64
maquinas_ativas          float64
horas_operacao_turno     float64
consumo_kwh              float64
dtype: object

 Ausentes
timestamp                 0
turno                     0
producao_unidades        51
temperatura_ambiente_c   50
maquinas_ativas          50
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5050 entries, 0 to 5049
Data columns (total 7 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   timestamp              5050 non-null   object 
 1   turno                  5050 non-null   object 
 2   producao_unidades      4999 non-null   float64
 3   temperatura_ambiente_c 5000 non-null   float64
 4   maquinas_ativas        5000 non-null   float64
 5   horas_operacao_turno   4999 non-null   float64
 6   consumo_kwh            5050 non-null   float64
dtypes: float64(5), object(2)
memory usage: 276.3+ KB
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Limpeza e análise de outliers

Remova duplicatas exatas; mantenha um registro de quantas foram removidas. Converta datas sem usá-las diretamente como preditores nesta introdução. Investigue valores improváveis antes de removê-los. O limite de 50 °C para temperatura ambiente é apenas uma regra didática de sinalização, não uma especificação do processo. Importante: valores ausentes dos preditores serão preenchidos com medianas aprendidas somente no treinamento, via Pipeline. Não calcular fillna(median()) no dataset completo antes do split.

```
# Limpeza e análise de outliers
df1 =df.drop_duplicates().copy() # varre o dataframe em busca de valores duplicados
df['timestamp'] = pd.to_datetime(df['timestamp'],errors='coerce')
print('Removidas:', len(df)-len(df1),"duplicadas")
print('Datas inválidas:', df['timestamp'].isna().sum())
print('Temperaturas acima de 50 ºC para investigar:',(df['temperatura_ambiente_c']>50).sum())
print('Consumo<=0',(df["consumo_kwh"]<=0).sum())

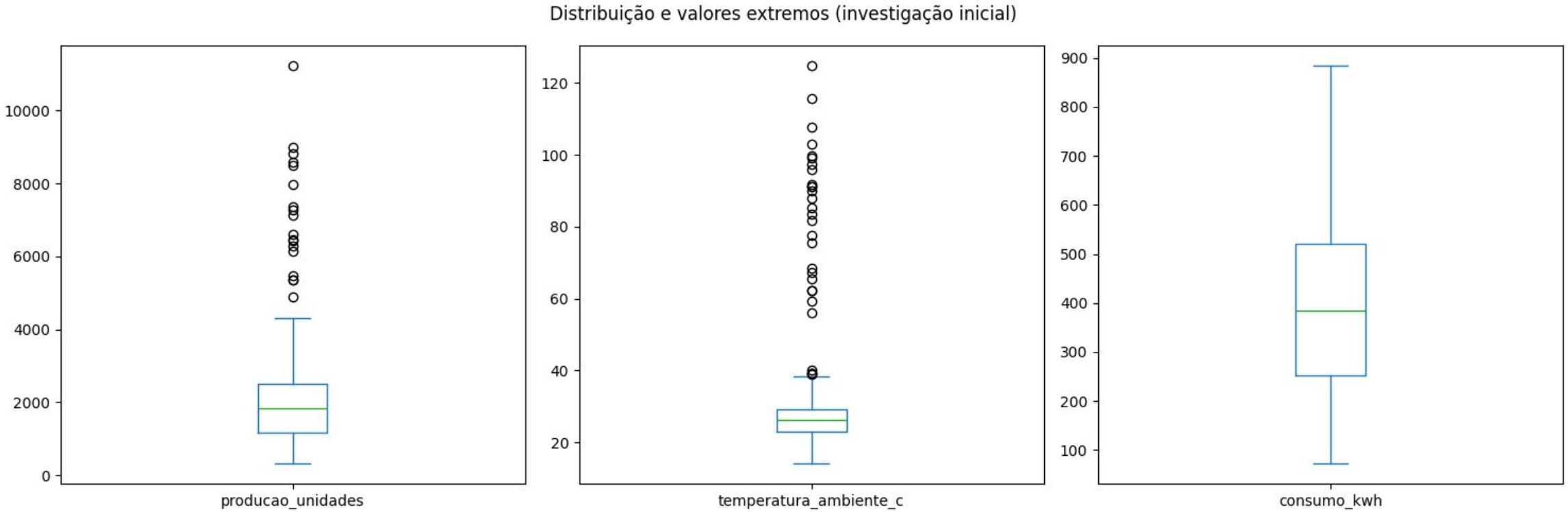
```

```
Removidas: 50 duplicadas
Datas inválidas: 0
Temperaturas acima de 50 ºC para investigar: 25
Consumo<=0 0
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

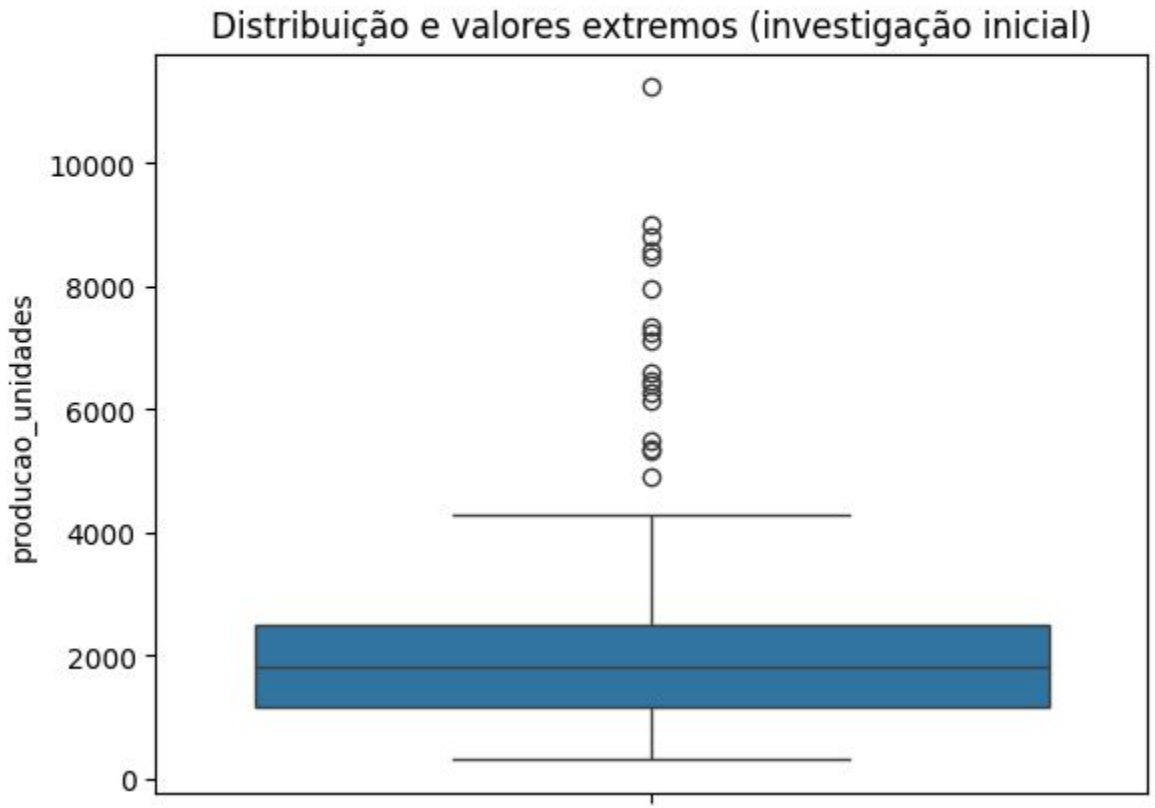
```
#Visualização
df[['producao_unidades','temperatura_ambiente_c','consumo_kwh']].plot(kind='box',subplots=True,layout=(1,3),figsize=(15,5),sharey=False)
plt.suptitle('Distribuição e valores extremos (investigação inicial)')
plt.tight_layout()
plt.show()
```



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
1 sns.boxplot(data=df['producao_unidades'])
2 plt.title('Distribuição e valores extremos (investigação inicial)')
3 plt.show()
```



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Introdução ao aprendizado supervisionado - Prever uma classe consumo alto ou baixo - Classificação: prever uma classe (consumo alto ou baixo).

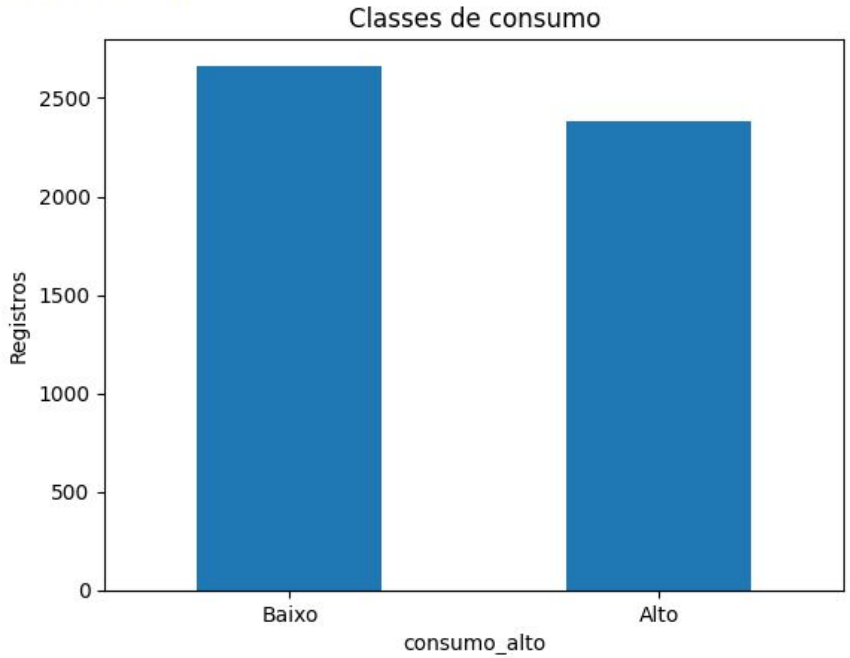
- Regressão: prever um número (ex.: kWh).
- Supervisionado: o rótulo correto está disponível durante o treino.
- Não supervisionado: buscamos padrões sem rótulos previamente definidos. Nesta atividade, alto consumo significa consumo_kwh > 400, e baixo significa <= 400. 400 kWh é um limite DIDÁTICO, não um critério de desempenho industrial. Na entrada do modelo, nunca incluir consumo_kwh, porque ele é a própria origem do rótulo.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```python
# Consumo é medido e serve apenas para criar o rótulo histórico
assert df["consumo_kwh"].notna().all()
df["consumo_alto"] = (df["consumo_kwh"] > 400).astype(int)
print(df["consumo_alto"].value_counts().rename(index={0:"Baixo",1:"Alto"}))
df["consumo_alto"].map({0:"Baixo",1:"Alto"}).value_counts().plot(kind="bar", title="Classes de consumo")
plt.xticks(rotation=0); plt.ylabel("Registros"); plt.show()
```

```
consumo_alto
Baixo    2666
Alto     2384
Name: count, dtype: int64
```



Exemplo 2 de Machine Learning e tratamento dos dados

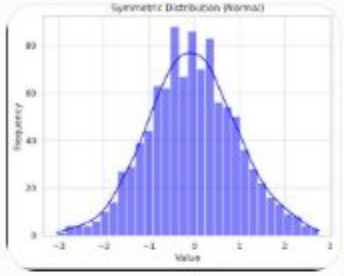
Tratamento dos dados ausentes

1. Quando utilizar média ou mediana?

Critério	Média (`mean`)	Mediana (`median`)
Distribuição aproximadamente simétrica	Recomendada	Também pode ser usada
Presença de outliers	Sensível	Mais robusta
Distribuição assimétrica	Pode ser distorcida	Geralmente preferível
Exemplo industrial	Temperatura estável	Consumo com picos extremos

Exemplo 2 de Machine Learning e tratamento dos dados

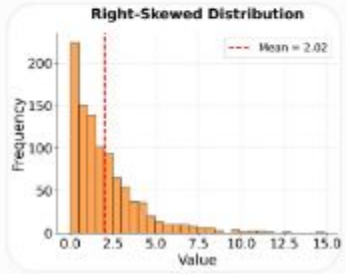
Tratamento dos dados ausentes



Distribuição aproximadamente simétrica

Os valores se concentram no centro. Média e mediana costumam ser próximas.

A média pode ser uma boa escolha para imputação.



Distribuição assimétrica

Os dados apresentam uma cauda mais longa para um dos lados.

A mediana geralmente é preferível, pois sofre menos influência de valores extremos.

Exemplo 2 de Machine Learning e tratamento dos dados

Verificando a distribuição dos dados

```python
import matplotlib.pyplot as plt

# Seleciona a variável
dados = df["producao_unidades"].dropna()

# Calcula média e mediana
media = dados.mean()
mediana = dados.median()

# Plota o histograma
plt.figure(figsize=(9, 5))

plt.hist(
    dados,
    bins=20,
    color="skyblue",
    edgecolor="black"
)

# Linha indicando a média
plt.axvline(
    media,
    color="red",
    linestyle="--",
    label=f"Média: {media:.2f}"
)

```

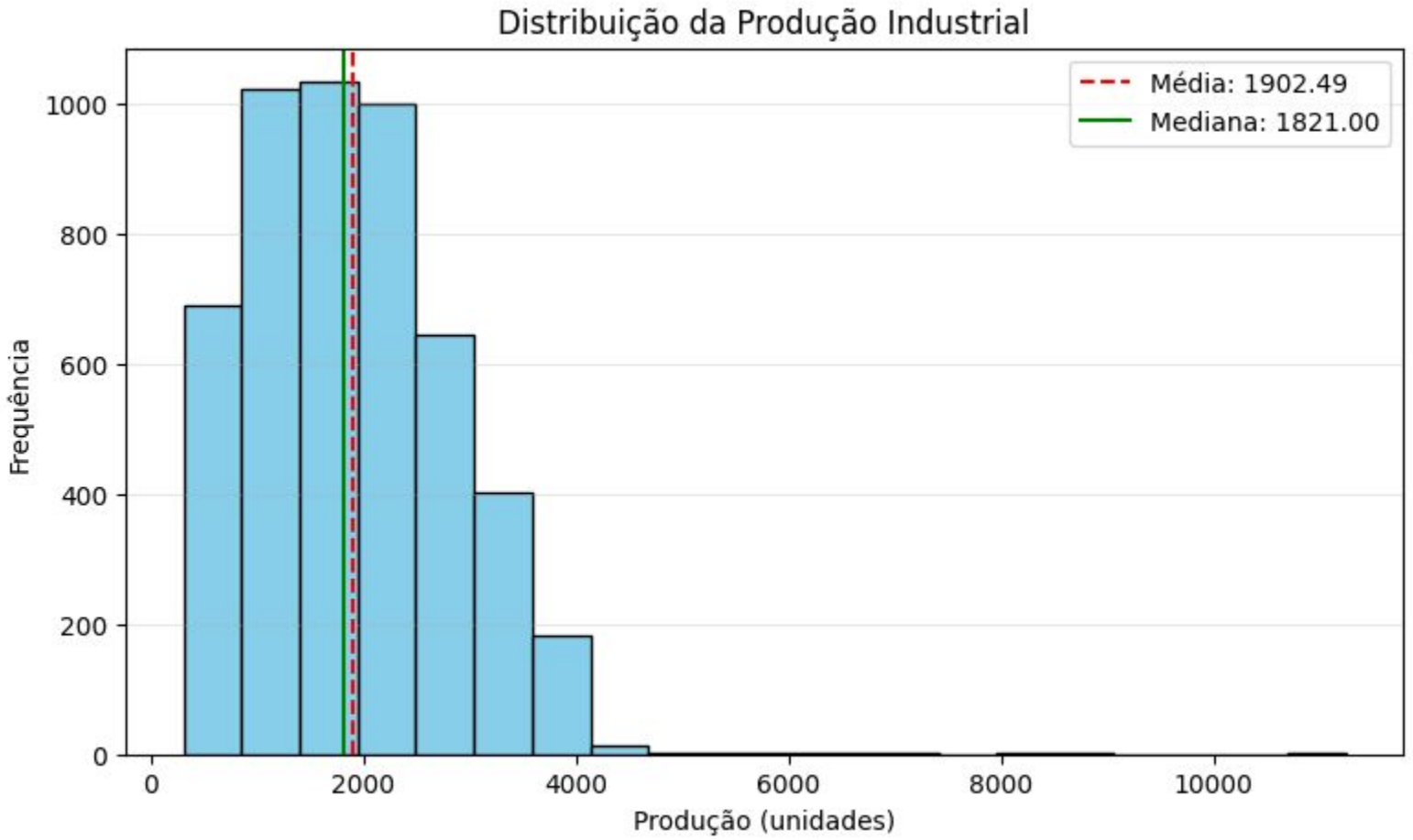
Exemplo 2 de Machine Learning e tratamento dos dados

Verificando a distribuição dos dados

```
27
28 # Linha indicando a mediana
29 plt.axvline(
30     mediana,
31     color="green",
32     linestyle="-",
33     label=f"Mediana: {mediana:.2f}"
34 )
35
36 plt.title("Distribuição da Produção Industrial")
37 plt.xlabel("Produção (unidades)")
38 plt.ylabel("Frequência")
39 plt.legend()
40 plt.grid(axis="y", alpha=0.3)
41
42 plt.show()
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Separar os atributos, treinar e prever Os preditores numéricos são produção, temperatura, máquinas ativas e horas de operação; turno é categórico. O ColumnTransformer aplica medianas aos dados numéricos e codifica o turno; o Pipeline aprende tudo sem acessar os valores do conjunto de teste.

[53] ✓ 0s

```
numericas = ["producao_unidades", "temperatura_ambiente_c", "maquinas_ativas", "horas_operacao_turno"]
categoricas = ["turno"]
atributos = numericas + categoricas
X = df[atributos]
y = df["consumo_alto"]

X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

preprocessamento = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), numericas),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categoricas)
])
modelo_consumo = Pipeline([
    ("preparacao", preprocessamento),
    ("arvore", DecisionTreeClassifier(max_depth=4, random_state=42))
])
modelo_consumo.fit(X_treino, y_treino)
previsoes = modelo_consumo.predict(X_teste)
print("Treino:", len(X_treino), "| Teste:", len(X_teste))
```

```
Treino: 4040 | Teste: 1010
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Avaliação do primeiro modelo Acurácia: proporção de previsões corretas. Precisão da classe alta: entre os turnos previstos como alto consumo, quantos de fato foram altos? Recall da classe alta: entre os turnos de alto consumo, quantos foram identificados? F1: equilíbrio entre precisão e recall. Uma matriz de confusão mostra acertos e erros. Em problemas reais, discuta qual tipo de erro é mais caro.

```python
def avaliar(nome, verdadeiro, previsto, rotulos=("Baixo", "Alto")):
    print(f"\n== {nome} ==")
    print(f"Acurácia: {accuracy_score(verdadeiro, previsto):.3f}")
    print(f"Precisão (classe 1): {precision_score(verdadeiro, previsto, zero_division=0):.3f}")
    print(f"Recall (classe 1): {recall_score(verdadeiro, previsto, zero_division=0):.3f}")
    print(f"F1 (classe 1): {f1_score(verdadeiro, previsto, zero_division=0):.3f}")
    print(classification_report(verdadeiro, previsto, target_names=rotulos, zero_division=0))
    ConfusionMatrixDisplay.from_predictions(verdadeiro, previsto, display_labels=rotulos, cmap="Blues")
    plt.title(nome); plt.show()

avaliar("Consumo ALTO/BAIXO", y_teste, previsoes)
```

```
== Consumo ALTO/BAIXO ==
Acurácia: 0.969
Precisão (classe 1): 0.974
Recall (classe 1): 0.960
F1 (classe 1): 0.967
              precision    recall  f1-score   support

       Baixo       0.96      0.98      0.97       533
        Alto       0.97      0.96      0.97       477

    accuracy                           0.97      1010
   macro avg       0.97      0.97      0.97      1010
```

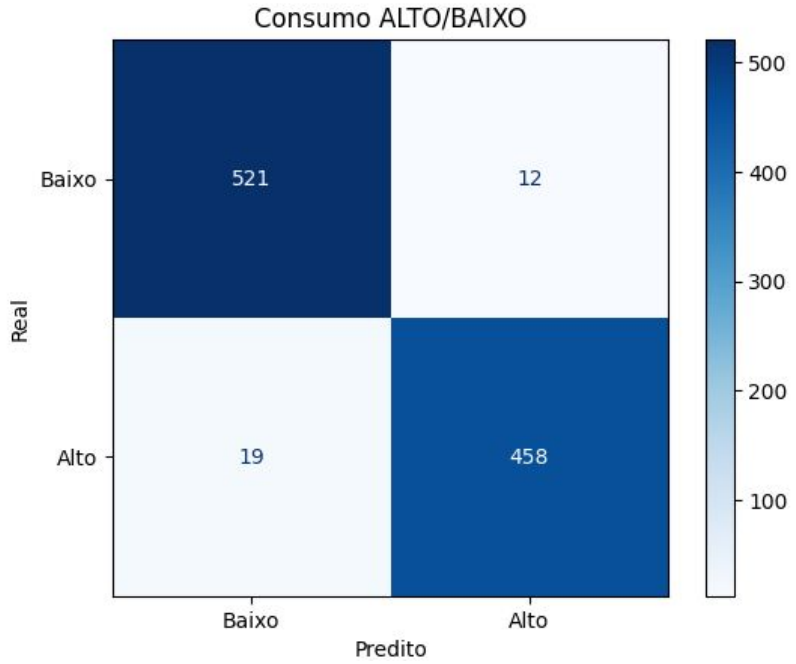
Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
== Consumo ALTO/BAIXO ==
Acurácia: 0.969
Precisão (classe 1): 0.974
Recall (classe 1): 0.960
F1 (classe 1): 0.967
              precision    recall  f1-score   support

       Baixo       0.96      0.98      0.97       533
        Alto       0.97      0.96      0.97       477

    accuracy                           0.97      1010
   macro avg       0.97      0.97      0.97      1010
weighted avg       0.97      0.97      0.97      1010
```



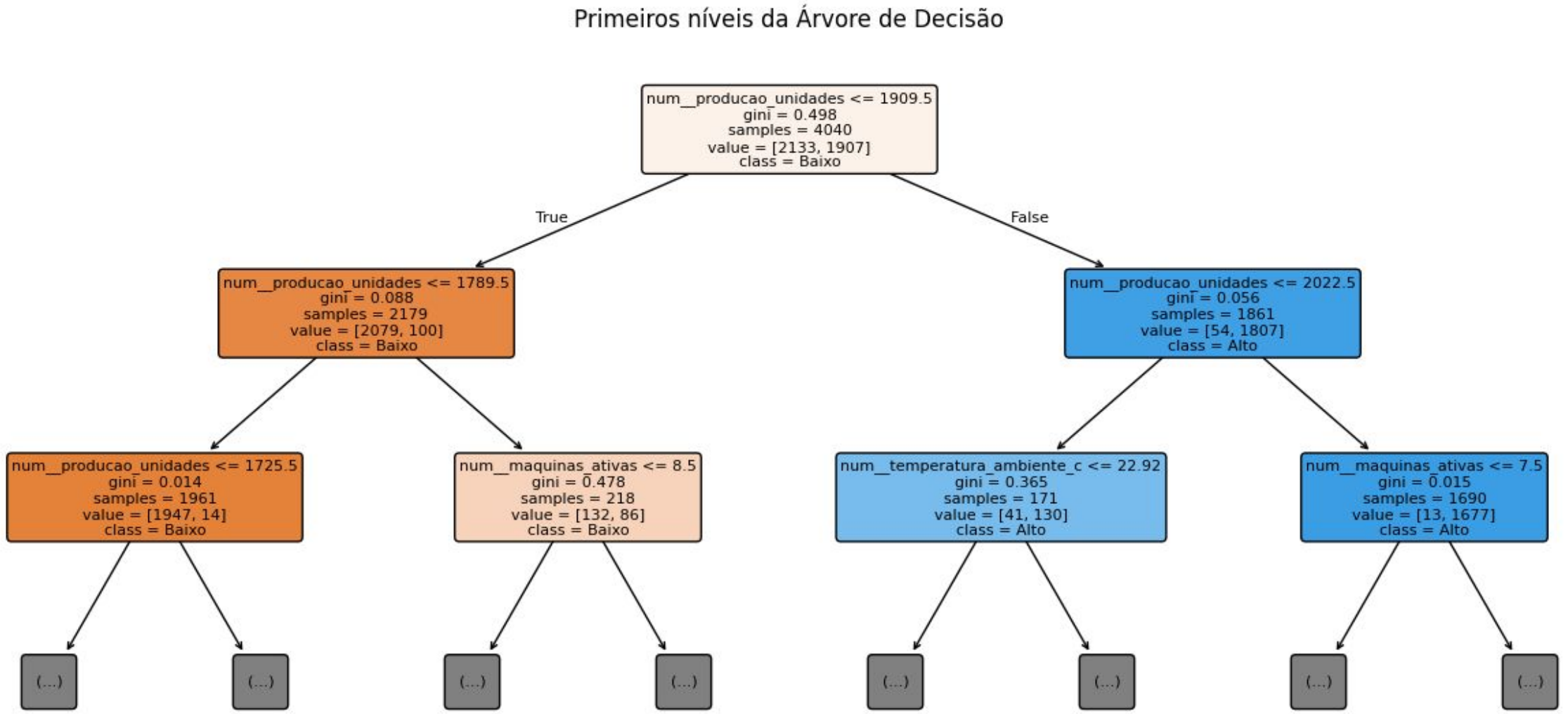
Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
# Visualização opcional: primeiros níveis da árvore
nomes = modelo_consumo.named_steps["preparacao"].get_feature_names_out()
plt.figure(figsize=(16,7))
plot_tree(modelo_consumo.named_steps["arvore"],
          feature_names=nomes, class_names=["Baixo", "Alto"],
          filled=True, rounded=True, fontsize=8, max_depth=2)
plt.title("Primeiros níveis da Árvore de Decisão"); plt.show()
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Segundo Classificador: Random Forest

Agora vamos treinar um modelo de **Random Forest** utilizando os mesmos dados de treino e a mesma estrutura de pré-processamento via `Pipeline`. Após o treinamento, utilizaremos a função `avaliar` para comparar as métricas com as da nossa primeira Árvore de Decisão.

```python
from sklearn.ensemble import RandomForestClassifier

# Criando o Pipeline com o preprocessamento existente e o classificador Random Forest
modelo_rf = Pipeline([
    ("preparacao", preprocessamento),
    ("floresta", RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42))
])

# Treinando o modelo
modelo_rf.fit(X_treino, y_treino)

# Fazendo as previsões no conjunto de teste
previsoes_rf = modelo_rf.predict(X_teste)

# Avaliando o desempenho do Random Forest
avaliar("Consumo ALTO/BAIXO (Random Forest)", y_teste, previsoes_rf)
```

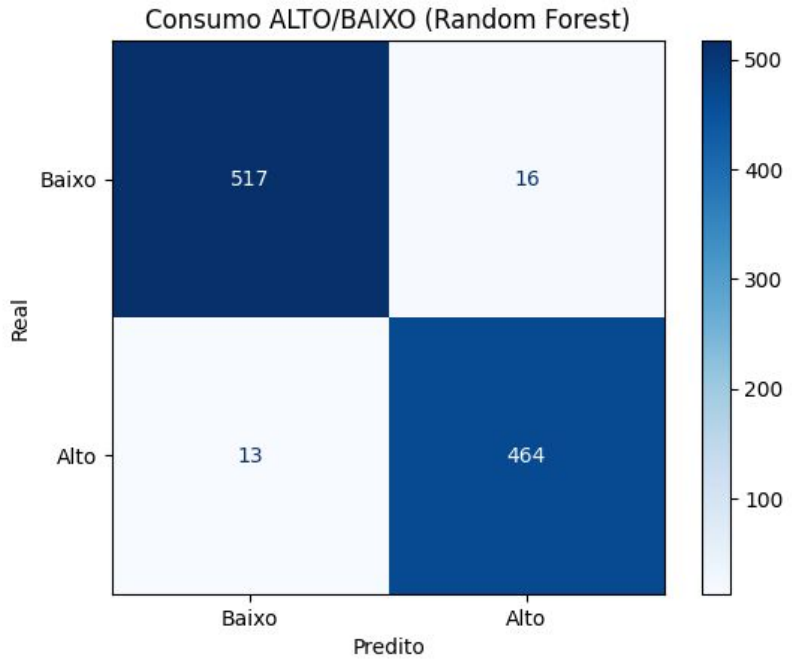
Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
== Consumo ALTO/BAIXO (Random Forest) ==
Acurácia: 0.971
Precisão (classe 1): 0.967
Recall (classe 1): 0.973
F1 (classe 1): 0.970
              precision    recall  f1-score   support

       Baixo       0.98      0.97      0.97       533
        Alto       0.97      0.97      0.97       477

    accuracy                           0.97      1010
   macro avg       0.97      0.97      0.97      1010
weighted avg       0.97      0.97      0.97      1010
```



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

O modelo Random Forest foi avaliado utilizando **1.010 registros de teste**, classificados em consumo Alto ou Baixo.

- **Acurácia (97,1%):** aproximadamente 97% dos registros foram classificados corretamente.
- **Precisão (96,7%):** das previsões de consumo Alto, aproximadamente 97% estavam corretas.
- **Recall (97,3%):** o algoritmo identificou aproximadamente 97% dos registros que realmente apresentavam consumo Alto.
- **F1-score (97,0%):** indica equilíbrio entre precisão e recall.
- **Support:** 533 registros de consumo Baixo e 477 de consumo Alto.

Conclusão: o Random Forest apresentou elevado desempenho na classificação do consumo energético, com resultados semelhantes nas duas classes.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Classificação do Consumo Energético — Random Forest

O algoritmo Random Forest foi utilizado para classificar o consumo energético industrial em duas categorias: **Alto (1) e Baixo (0)**.

Variáveis utilizadas: produção, temperatura ambiente, quantidade de máquinas ativas, horas de operação e turno.

Resultados obtidos:

- Acurácia: 97,1%.
- Precisão: 96,7%.
- Recall: 97,3%.
- F1-score: 97,0%.

Interpretação: o modelo apresentou elevado desempenho na classificação dos registros de teste, utilizando características operacionais da indústria para identificar padrões associados ao nível de consumo energético.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Comparação dos Modelos de Classificação

Os algoritmos Árvore de Decisão e Random Forest foram utilizados para classificar o consumo energético industrial em Alto e Baixo, a partir de variáveis operacionais.

Resultados:

- Árvore de Decisão: acurácia de 96,9%, com 31 classificações incorretas.
- Random Forest: acurácia de 97,1%, com 29 classificações incorretas.
- A Árvore de Decisão apresentou maior precisão na identificação de consumo Alto (97,4%).
- O Random Forest apresentou maior recall para consumo Alto (97,3%), identificando 464 dos 477 casos dessa categoria.

Conclusão: ambos os algoritmos apresentaram elevado desempenho no conjunto de teste. O Random Forest obteve uma pequena vantagem na acurácia e identificou mais registros de consumo Alto, enquanto a Árvore de Decisão produziu menos falsos alertas dessa categoria.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Em uma indústria, é mais prejudicial classificar um consumo baixo como alto ou deixar de identificar um consumo realmente alto?

Diferença entre programação tradicional e Machine Learning

- Programação tradicional:
- Dados + Regras programadas → Resultado
- Machine Learning:
- Dados + Resultados conhecidos → Treinamento → Modelo aprendido
- Depois:
- Novos dados + Modelo aprendido → Previsão

Link material

Material disciplina

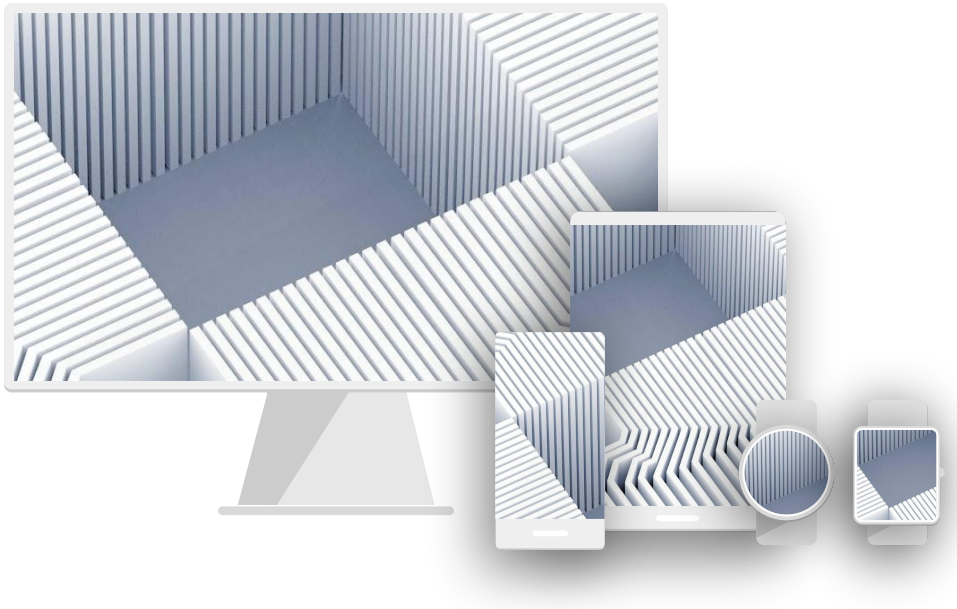
Obrigado !

Prof. Me Daniel Vieira

Email: danielvieira2006@gmail.com

Linkedin: Daniel Vieira

Instagram: Prof daniel.vieira95



UNISENAI
Inovação no
Ensino Superior
para a Indústria